We will assume a two-byte packed BCD representation. The first byte represents two denary digits for the whole part of the number and the second byte represents two denary digits for the fractional part. If the two values are $0.26 and $0.85 then the result of the addition should be $1.11. This would involve a carry from the first decimal place to the whole number 1. However, applying simple binary addition of the BCD codes would produce the result shown in Figure 1.02.

The additions for the fractional parts have produced values corresponding to the denary values 10 and 11 but a BCD value is supposed to be a single digit. The error has resulted in no carry to the whole number column.

We need the processor to recognise that an impossible value has been produced and apply a method to correct this. The solution is to add 0110 whenever the problem is detected. This is illustrated in Figure 1.03.

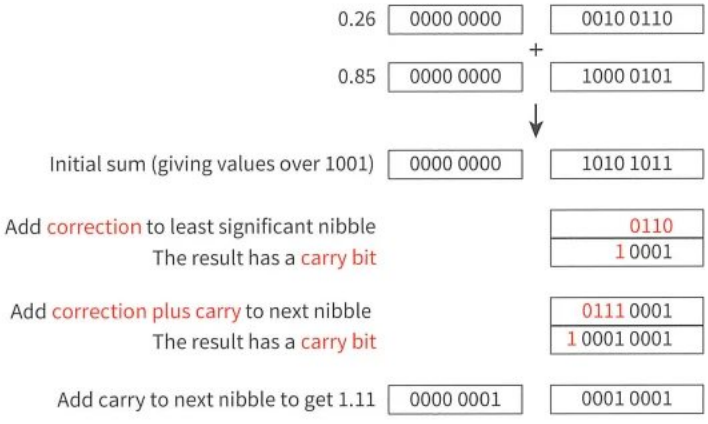


Figure 1.03 Use of the correction value to perform BCD addition

The steps shown in Figure 1.03 are as follows.

- Starting with the least significant nibble, adding 0110 to 0101 gives 1011 which is recognised as being incorrect.
- The 0110 correction value is added to produce 10001.
- The 0001 is stored and the leading 1 is carried to the next nibble.
- In the first decimal position adding 0100 to 1000 then adding the carry bit 1 gives 1011 which is recognised as being incorrect.
- The 0110 correction is added to produce 10001.
- The 0001 is stored and the leading 1 is carried to the next nibble.

In this example the two whole number nibbles have zero values so adding these has no effect.

1.04 Internal coding of text

To store text in a computer, we need a coding scheme that provides a unique binary code for each distinct individual component item of the text. Such a code is referred to as a character code. There have been many different examples of character coding schemes throughout the history of computing.

ASCII code

The scheme which has been used for the longest time is the ASCII (American Standard Code for Information Interchange) coding scheme. The 7-bit version of the code (often referred to as US ASCII) was standardised many years ago by ANSI (American National Standards Institute). The codes are always presented in a table. Table 1.09 shows an edited version of a typical table. The first column contains the binary code which would be stored in one byte, with the most significant bit set to zero and the remaining bits representing the character code. The second column shows the hexadecimal equivalent.

Binary code	Hexadecimal equivalent	Character	Description
00000000	00	NUL	Null character
00000001	01	SOH	Start of heading
00100000	20		Space
00100001	23	#	Number
00110000	30	0	Zero
00110001	31	1	One
01000001	41	A	Uppercase A
01000010	42	B	Uppercase B
01100001	61	a	Lowercase a
01100010	62	b	Lowercase b

Table 1.09 Some examples of ASCII codes stored in one byte with the remaining, most significant bit set to zero

A full table would show the 2^7 (128) different codes available for a 7-bit code.

TIP

Do not try to remember any of the individual codes

You need to remember these key facts about the ASCII coding scheme.

- A limited number of the codes represent non-printing or control characters; these were introduced to assist in data transmission or for data handling at a computer terminal.
- The majority of the codes are for characters that would be found in an English text and which are available on a standard keyboard.
- These include upper- and lower-case letters, punctuation marks, denary digits and arithmetic symbols.
- The codes for numbers and for letters are in sequence so that, for example, if 1 is added to the code for seven, the code for eight is produced.
- The codes for the upper-case letters differ from the codes for the corresponding lower-case letters only in the value of bit 5, which allows a simple conversion from upper to lower case or the reverse. (Don't forget that the least significant bit is bit 0.)

Note that this coding for numbers is exclusively for use in the context of stored, displayed or printed text. All of the other coding schemes for numbers are for internal use in a computer system and would not be used in a text.

Although a standard version of ASCII has been created, different versions of 7-bit ASCII are tailored to different software or different countries. Mostly, the coding for the printable

characters has remained unchanged. A notable exception was the use in some countries of the code 00100001 to represent a currency symbol rather than #. However, because most of the control characters became of limited use, there were versions of ASCII that used these codes to produce small graphic icons. For example, the code 00000001 would show ☺.

Extended ASCII is a code that uses all eight bits in a byte. The most used standardised version is often referred to as ISO Latin-1. The name Latin-1 reflects the fact that many of the new character definitions are for accented or otherwise modified alphabetic characters found in European languages, for example Ç or ü. As with the 7-bit code, there are many variations of the standard code.

Question 1.02

Many years ago, a byte was defined as six bits. If a character was to be represented by one byte, which characters would you expect to be representable and which ones would you expect to be unavailable?

Unicode

Although ASCII codes are widely used, they do not cover all the characters needed for some uses. For this reason, new coding schemes have been developed and continue to be developed further. The discussion here describes one of the Unicode schemes. It should be noted that Unicode codes have been developed in tandem with the Universal Character Set (UCS) scheme, standardised as ISO/IEC 10646.

The aim of Unicode is to be able to represent any possible text in code form. In particular, this includes all languages in the world. The most popular version of Unicode which is discussed here is named UTF-8. The inclusion of 8 in the name indicates that this version of the standard includes codes defined by one byte in addition to codes using two, three and four bytes.

Figure 1.04 shows the structure of the codes. The 1 byte code reproduces 7-bit ASCII. Because the byte has the most significant bit set to 0 there can be no confusion with any byte which is part of a multiple byte code. Note that for the two-byte, three-byte and four-byte representations all continuing bytes have the two most significant bits set to 10. Whenever a byte has the most significant bits set to 11 there will be at least one continuation byte following.

0???????			
110?????	10??????		
1110????	10??????	10??????	
11110???	10??????	10??????	10??????

Figure 1.04 Byte formats for Unicode UTF-8

The number of codes available is determined by the number of bits that are not pre-defined by the format. For example, there are eleven bits free to identify codes in the 2-byte format. This allows $2^{11} = 2048$ different codes.

Unicode has its own special terminology and symbolism. A character code is referred to as a 'code point'. In any documentation a code point is identified by U+ followed by a 4-digit hexadecimal number. The code points U+0000 to U+00FF define characters which are a duplicate of those in the standard Latin-1 scheme. The binary codes corresponding to U+0000 to U+007F use one byte only and range from 00000000 through to 01111111. Then the binary codes for U+0080 to U+00FF require two bytes and range from 11000000 for the first byte followed by 10000000 for the second byte through to 11000001 followed by 10111111.

1.05 Images

Images can be stored in a computer system for the eventual purpose of displaying the image on a screen or for presenting it on paper, usually as a component of a document. Such an image can be created by using an appropriate graphics package. Alternatively, when an image already exists independently of the computer system, the image can be captured by using photography or by scanning.

Vector graphics

In an image that is created by a drawing package or a computer-aided design (CAD) package each component is an individual **drawing object**. The image is then stored, usually as a **vector graphic** file.

We do not need to consider how an image of this type would be created. We do need to consider how the data is stored after the image has been created. A vector graphic file contains a **drawing list**. The list contains a command for each object included in the image. Each command has a list of attributes, each attribute defines a **property** of the object. The properties include the basic geometric data such as, for a circle, the position of the centre and its radius. In addition, properties are defined such as the thickness and style of a line, the colour of a line and the colour that fills the shape. An example of what could be created as a vector graphic file is shown in Figure 1.05.

KEY TERMS

Vector graphic: a graphic consisting of drawing objects defined in a drawing list

Drawing object: a component defined by geometric formulae and associated properties

Drawing list: contains one set of values for each drawing object

Property: defines one aspect of the appearance of the drawing object

TASK 1.05

Construct a partial drawing list for the graphic shown in Figure 1.05. You can take measurements from the image and use the bottom left corner of the box as the origin of a coordinate system. You can invent your own format for the drawing list.

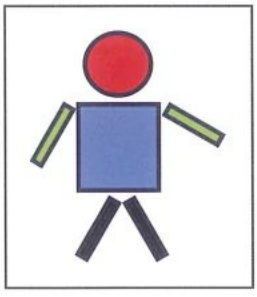


Figure 1.05 A simple example of a vector graphic image

The most important property of a vector graphic image is that the dimensions of the objects are not defined explicitly but instead are defined relative to an imaginary drawing canvas. In other words, the image is scalable. Whenever the image is to be displayed the file is read, the appropriate calculations are made and the objects are drawn to a suitable scale. If the user then requests that the image is redrawn at a larger scale the file is read again and another set

of calculations are made before the image is displayed. This avoids image distortion, such as the image appearing squashed or stretched.

Note that a vector graphic file can only be displayed directly on a graph plotter, which is an expensive specialised piece of hardware. For the image to appear correctly on other types of display, the vector graphic file often has to be converted to a bitmap.

Bitmaps

Most images do not consist of geometrically defined shapes, so a vector graphic representation is inappropriate. Instead, generally an image is stored as a bitmap. Typical uses are when capturing an existing image by scanning or perhaps by taking a screen-shot. Alternatively, an image can be created by using a simple graphics package.

The fundamental concept underlying the creation of a bitmap file is that the **picture element (pixel)** is the smallest identifiable component of a bitmap image. The image is stored as a two-dimensional matrix of pixels. The pixel itself is a very simple construct; it has a position in the matrix and it has a colour. It does not matter whether each pixel is a small rectangle, a small circle or a dot.

The scheme used to represent the colour has to be defined. The simplest option is to use one bit to represent the colour, so that the pixel is either black or white. Storage of the colour in four bits allows simple greyscale colouring. At least eight bits per pixel are necessary to provide a sufficient range of colours to provide a reasonably realistic representation of any image. The number of bits per pixel is sometimes referred to as the **colour depth**.

KEY TERMS

Picture element (pixel): the smallest identifiable component of a bitmap image, defined by just two properties: its position in the bitmap matrix and its colour

Colour depth: the number of bits used to represent one pixel

An alternative definition is the **bit depth**. Although these terms are sometimes used interchangeably, bit depth is best defined as the number of bits used to store each of the red, green and blue primary colours in the RGB colour scheme.

A colour depth of 8 bits per pixel provides 256 different colours. A bit depth of 8 bits per primary colour provides $256 \times 256 \times 256 = 16\,777\,216$ different colours. The eye cannot distinguish this number of different colours. However, this many are needed if an image contains areas of gradually changing colour such as in a picture of the sky. If a lower bit depth is used the image will show bands of colour.

We also need to decide which resolution to use for the image, which can be represented as the product of the number of pixels per row times the number of rows. When considering resolution it is important to distinguish between an **image resolution**, as defined in a bitmap file, and a **screen resolution** for a particular monitor screen that might be used to display the image. Both of these have to be considered if a screen display is being designed.

KEY TERMS

Bit depth: the number of bits used to represent each of the red, green and blue colours

Image resolution: the number of pixels in the bitmap file defined as the product of the width and the height values

Screen resolution: the product of width and height values for the number of pixels that the screen can display

A bitmap file does not define the physical size of a pixel or of the whole image. When the image is scaled the number of pixels in it does not change. If a well-designed image is presented on a suitable screen the human eye cannot distinguish the individual pixels. However, if the image is magnified too far the individual pixels will be seen. This is illustrated in Figure 1.06 which shows an original small image, a magnified version of this small image and a larger image created with a more sensible, higher resolution.



a b c

Figure 1.06 (a) a bitmap logo; (b) an over-magnified version of the image; (c) a sensible larger version

File size is always an issue with an image file. A large file occupies more memory space and takes longer to display or to be transmitted across a network. Usually, a vector graphic file uses considerably less memory space than a corresponding bitmap file.

You can calculate the size of a bitmap graphic knowing the resolution and the colour depth. As an example, consider that a bitmap graphic is needed to fill a laptop screen where the resolution is 1366 by 768. If we want colour depth of 24 then the number of bits we need is:

$$1366 \times 768 \times 24 = 25\,178\,112 \text{ bits}$$

The result of this calculation shows the number of bits, but a size is always quoted as a number of bytes or multiples of bytes. For our bitmap graphic:

25 178 112 bits = 25 178 112 ÷ 8 = 3 147 264 bytes

= 3 147 264 ÷ 1024 = 3073.5 kibibytes (3073.5 KiB)

= 3073.5 ÷ 1024 = approximately 3 MiB

Note that this calculation has assumed that the colour depth specifies the total number of bits used to define each pixel. If the information given was that the bit depth was eight, then the calculation would use 8 + 8 + 8 for the number of bits per pixel.

WORKED EXAMPLE 1.04

You have been asked to calculate a value for the minimum size of a bitmap file. The bitmap is to use a bit depth of 8 and the bitmap is to be printed with 72 dpi (dots per inch) and to have dimensions 5 inches by 3 inches.

We use the information provided about the colour depth or the bit depth to give the number of bits per pixel. In this case the bit depth is 8, which means 8 bits for each of the RGB components, so 24 bits are needed for one pixel.

Let's state that 72 dpi means 72 pixels per inch.

(Continued)

So, the number of pixels per row is 5 × 72 = 360

And the number of pixels per column is 3 × 72 = 216

Therefore, the total number of pixels is 360 × 216 = 77 760

The total number of bits is this value multiplied by 24. However, we want the size in bytes not bits, so we multiply by 3 because there are 8 bits in a byte. So, we get:

77 760 × 3 = 233 280 bytes.

We can quote this in kibibytes by dividing by 1024:

233 280 / 1024 = 227.8 KiB

A bitmap file has to store the pixel data that defines the graphic, but the file must also have a **file header** that contains information on how the graphic has been constructed. Because of this, the bitmap file size is larger than the size of the graphic alone. At the very least the header will define the colour depth or bit depth and the resolution.



KEY TERM

File header: a set of bytes at the beginning of a bitmap file which identifies the file and contains information about the coding used

The following are considerations when justifying the use of either a bit map or a vector graphic for a specific task.

- A vector graphic is chosen if a diagram is needed to be constructed for part of an architectural, engineering or manufacturing design.
- If a vector graphic file has been created but there is a need to print a copy using a laser or inkjet printer the file has first to be converted to a bitmap.
- A digital camera automatically produces a bitmap.
- A bitmap file is the choice for insertion of an image into a document, publication or web page.

1.06 Sound

Natural sound consists of variations in pressure which are detected by the human ear. A typical sound contains a large number of individual waves, each with a defined frequency. The result is a wave form in which the amplitude of the sound varies in a continuous but irregular pattern.

If we want to store sound or transmit it electronically the original analogue sound signal has to be converted to a binary code. The measured sound values are input to a sound encoder which has two components. The first is a band-limiting filter. This is needed to remove high-frequency components. A human ear cannot detect these very high frequencies and they could cause problems for the coding if not removed. The other component in the encoder is an analogue-to-digital converter (ADC) which converts the **analogue data** to **digital data**.

Figure 1.07 shows the **sampling** operation of the ADC. The amplitude of the wave (the red line) has to be sampled at regular intervals. The blue vertical lines indicate the sampling times. The amplitude cannot be measured exactly; instead the amplitude is approximated